6.4.7 Question:

EXERCISE | 6.7.4

(a) Draw the 11-item hash table resulting from hashing the keys 12, 44, 13, 88, 23, 94, 11, 39, 20, 16, and 5, using the hash function $h(k) = (2k + 5) \bmod 11$ and assuming collisions are handled by chaining.

Feedback?

6.4.7 Solution:

Since the hash function is $h(k) = (2k + 5)\, mod\, 11$, calculate each hash value:

- Key 12: $h(12) = (2 \times 12 + 5)\, mod\, 11 = 29\, mod\, 11 = 7$
- Key 44: $h(44) = (2 \times 44 + 5)\, mod\, 11 = 93\, mod\, 11 = 5$
- Key 13: $h(12) = (2 \times 13 + 5)\, mod\, 11 = 31\, mod\, 11 = 9$
- Key 88: $h(88) = (2 \times 88 + 5)\, mod\, 11 = 181\, mod\, 11 = 5$
- Key 23: $h(23) = (2 \times 23 + 5)\, mod\, 11 = 51\, mod\, 11 = 7$
- Key 94: $h(94) = (2 \times 94 + 5)\, mod\, 11 = 193\, mod\, 11 = 6$
- Key 11: $h(11) = (2 \times 11 + 5)\, mod\, 11 = 27\, mod\, 11 = 5$
- Key 39: $h(39) = (2 \times 39 + 5)\, mod\, 11 = 83\, mod\, 11 = 6$
- Key 20: $h(20) = (2 \times 20 + 5)\, mod\, 11 = 45\, mod\, 11 = 1$
- Key 16: $h(16) = (2 \times 16 + 5)\, mod\, 11 = 37\, mod\, 11 = 4$
- Key 5: $h(5) = (2 \times 5 + 5)\, mod\, 11 = 15\, mod\, 11 = 4$

Then the 11-item hash table (collisions are handled by chaining):

0	
1	•→ 20
2	
3	
4	•→ 16 − 5
5	•→ 44 − 88 − 11
6	•→ 94 − 39
7	•→ 12 − 23
8	
9	•→ 13
10	

6.7.7 Question:

EXERCISE | 6.7.7

(a) What is the result of Exercise 6.7.4 assuming collisions are handled by double hashing using a secondary hash function $h'(k) = 7 - (k \bmod 7)$?

Feedback?

6.7.7 Solution:

Since the primary hash function is $h(k) = (2k + 5)\, mod\, 11$, the secondary hash function is $h'(k) = 7 - (k\, mod\, 7)$, the probing position is $(i + j \cdot d(k))\, mod\, N,\ for\ i = h(k),\ j = 0, 1, \ldots, N - 1$, which is $(h(k) + j \cdot h'(k))\, mod\, 11$, then calculate each hash value:

- Key 12:
  - $h(12) = (2 \times 12 + 5)\, mod\, 11 = 29\, mod\, 11 = 7$
  - 7 is empty, insert 12 at index 7.
- Key 44:
  - $h(44) = (2 \times 44 + 5)\, mod\, 11 = 93\, mod\, 11 = 5$
  - 5 is empty, insert 44 at index 5.
- Key 13:
  - $h(12) = (2 \times 13 + 5)\, mod\, 11 = 31\, mod\, 11 = 9$
  - 9 is empty, insert 13 at index 9.
- Key 88:
  - $h(88) = (2 \times 88 + 5)\, mod\, 11 = 181\, mod\, 11 = 5$, collision with 44
  - $h'(88) = 7 - (88\, mod\, 7) = 7 - 4 = 3$
  - $j = 1$: $(h(k) + j \cdot h'(k))\, mod\, 11 = (5 + 1 \cdot 3)\, mod\, 11 = 8$
  - 8 is empty, insert 88 at index 8.
- Key 23:
  - $h(23) = (2 \times 23 + 5)\, mod\, 11 = 51\, mod\, 11 = 7$, collision with 12
  - $h'(23) = 7 - (23\, mod\, 7) = 7 - 2 = 5$
  - $j = 1$: $(h(k) + j \cdot h'(k))\, mod\, 11 = (7 + 1 \cdot 5)\, mod\, 11 = 1$
  - 1 is empty, insert 23 at index 1.
- Key 94:
  - $h(94) = (2 \times 94 + 5)\, mod\, 11 = 193\, mod\, 11 = 6$
  - 6 is empty, insert 94 at index 6.
- Key 11:
  - $h(11) = (2 \times 11 + 5)\, mod\, 11 = 27\, mod\, 11 = 5$, collision with 44
  - $h'(11) = 7 - (11\, mod\, 7) = 7 - 4 = 3$
  - $j = 1$: $(h(k) + j \cdot h'(k))\, mod\, 11 = (5 + 1 \cdot 3)\, mod\, 11 = 8$, occupied by 88
    $j = 2$: $(h(k) + j \cdot h'(k))\, mod\, 11 = (5 + 2 \cdot 3)\, mod\, 11 = 0$
  - 0 is empty, insert 11 at index 0.
- Key 39:
  - $h(39) = (2 \times 39 + 5)\, mod\, 11 = 83\, mod\, 11 = 6$, collision with 94

- $h'(39) = 7 - (39 \, mod \, 7) = 7 - 4 = 3$
- $j = 1$: $(h(k) + j \cdot h'(k)) \, mod \, 11 = (6 + 1 \cdot 3) \, mod \, 11 = 9$, occupied by 13
  $j = 2$: $(h(k) + j \cdot h'(k)) \, mod \, 11 = (6 + 2 \cdot 3) \, mod \, 11 = 1$, occupied by 23
  $j = 3$: $(h(k) + j \cdot h'(k)) \, mod \, 11 = (6 + 3 \cdot 3) \, mod \, 11 = 4$
- 4 is empty, insert 39 at index 4.

- Key 20:
  - $h(20) = (2 \times 20 + 5) \, mod \, 11 = 45 \, mod \, 11 = 1$, collision with 23
  - $h'(20) = 7 - (20 \, mod \, 7) = 7 - 6 = 1$
  - $j = 1$: $(h(k) + j \cdot h'(k)) \, mod \, 11 = (1 + 1 \cdot 1) \, mod \, 11 = 2$
  - 2 is empty, insert 20 at index 2.
- Key 16:
  - $h(16) = (2 \times 16 + 5) \, mod \, 11 = 37 \, mod \, 11 = 4$, collision with 39
  - $h'(16) = 7 - (16 \, mod \, 7) = 7 - 2 = 5$
  - $j = 1$: $(h(k) + j \cdot h'(k)) \, mod \, 11 = (4 + 1 \cdot 5) \, mod \, 11 = 9$, occupied by 13
    $j = 2$: $(h(k) + j \cdot h'(k)) \, mod \, 11 = (4 + 2 \cdot 5) \, mod \, 11 = 3$
  - 3 is empty, insert 16 at index 3.
- Key 5:
  - $h(5) = (2 \times 5 + 5) \, mod \, 11 = 15 \, mod \, 11 = 4$, collision with 39
  - $h'(5) = 7 - (5 \, mod \, 7) = 7 - 5 = 2$
  - $j = 1$: $(h(k) + j \cdot h'(k)) \, mod \, 11 = (4 + 1 \cdot 2) \, mod \, 11 = 6$, occupied by 94
    $j = 2$: $(h(k) + j \cdot h'(k)) \, mod \, 11 = (4 + 2 \cdot 2) \, mod \, 11 = 8$, occupied by 88
    $j = 3$: $(h(k) + j \cdot h'(k)) \, mod \, 11 = (4 + 3 \cdot 2) \, mod \, 11 = 10$
  - 10 is empty, insert 5 at index 10.

Then the 11-item hash table (collisions are handled by double hashing using a secondary hash function):

0 •→ 11
1 •→ 23
2 •→ 20
3 •→ 16
4 •→ 39
5 •→ 44
6 •→ 94
7 •→ 12
8 •→ 88
9 •→ 13
10 •→ 5

6.7.15 Question:

EXERCISE | 6.7.15

(a) A ***multimap*** is data structure that allows for multiple values to be associated with the same key. It has a `put`(k, v) method, which inserts an item with key k and value v even if there is already an item with key k (but not the same key-value pair), and a `FindAll` (k) method, which returns all the values that have the key k. Describe a scheme that implements a multimap so that the `put`(k, v) method runs in $O(1)$ expected time and the `FindAll`(k) method runs in $O(1 + s)$ time, where s is the number of values with key k.

Feedback?

6.7.15 Solution:

We can use a Hash Table with chaining, and store a list of values per key:

- Use a Hash Table A of size N, with hash function h.
- Each bucket $A[h(k)]$ stores a linked list of entries.
- Each entry = $(key,\ values)$, where $values$ is a linked list.

```
Structure Entry
    key
    values          // linked list of values

Algorithm put(k, v)
    Input: A key k and value v
    Output: Update table that includes item (k, v); If there already exists a list with key k,
            add the value v to the list.

    i ← h(k)

    for each Entry e in A[i] do      // if found the key, add v to the Entry.values list
        if e.key = k then
            append v to e.values
            return

    Create a new Entry e             // if not found, create new entry add into hash table
    e.key ← k
    e.values ← empty list
    append v to e.values
    insert e into A[i]

Algorithm FindAll(k)
    Input: A key k
    Output: All values that have the key k, or empty list if no such key exists

    i ← h(k)
```

```
for each Entry e in A[i] do      // if found the key, return the Entry.values list
        if e.key = k then
                return e.values

return empty list                // if not found
```

6.7.19 Question:

EXERCISE | 6.7.19

(a) In a ***double-entry accounting*** system, every business transaction has to be entered as two separate transactions, in different two accounts, once as a ***debit*** and once as a ***credit***. For example, if a business borrows $10,000 from a bank, the business should enter a debit of $10,000 to its Cash account a credit of $10,000 to its Notes Payable account. In order to be in balance, every debit in such a system must have a matching credit. Describe an efficient algorithm to test if a double-entry accounting system is in balance. What is the running time of your method in terms of n, the number of business transactions?

Feedback?

6.7.19 Solution:

We can use a hash table to check the balance for every business transaction:

- Initialize an empty Hash Table, where:
  - The key is the transactionID
  - The value is the net balance
- For each the entries of n in the business transactions:
  - If the entry is a debit, add the amount to the balance associated with the transactionID in the hash table
  - If the entry is a credit, subtract the amount from the balance associated with the transactionID in the hash table
- After process all entries, check the net balance of each key in the hash table:
  - If every value is 0, then the system is in balance.
  - If any value is non-zero, then the specific transaction is out of balance.

The running time is $O(n)$. Since we process each transaction once, and there are n entries, which need n hash table operation (insert, lookup, update). Each operation expected $O(1)$ time, so need $O(n)$ time. The final check pass over all entries, take $O(n)$ time. So the total running time is $O(n)$.